

MAIE 5532: Machine Learning Systems

Week 5 – Distributed Training

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lfteris NTAFL0S, PhD

The Hong Kong University of Science and Technology

Week 5 Overview

Distributed Training: Scaling ML Across Heterogeneous Devices

Learning Objectives:

- Master distributed training strategies from data centers to edge devices
- Analyze privacy-preserving distributed learning approaches
- Design secure federated learning systems
- Evaluate communication-efficient protocols for resource-constrained environments

Required Readings:

- [Book 1 - Volume 2: Distributed Training](#)
- [Book 1 - Volume 2: Collective Communication](#)
- [Book 1 - Volume 2: Fault Tolerance](#)
- [Book 1 - Volume 2: Edge Intelligence](#)
- [Book 2: Chapter 20: Privacy, Security, and Deployment](#)



Today's Structure

Session 1: Classical Distributed Training

- Data, model, pipeline, tensor, and hybrid parallelism
- Collective communication and topology-aware scaling
- Communication bottlenecks and fault-aware execution

Session 2: Federated Learning Systems

- FedAvg, client scheduling, non-IID data, and personalization
- Communication-efficient client updates
- Performance across heterogeneous edge devices

Session 3: Privacy, Security, and Deployment

- Privacy attacks, differential privacy, and secure aggregation
- Reliability under churn and malicious behavior
- Governance, deployment safety, and real-world system design

Activities

1. Estimate all-reduce cost for ResNet-50
2. Design a federated smart-keyboard protocol
3. Review a privacy-preserving healthcare deployment

The Distributed Training Challenge

Why Distributed Training Matters

Model memory grows quickly

For a model with P parameters:

Parameter memory = $P \times \text{bytes per parameter}$

Example: **175B parameters**

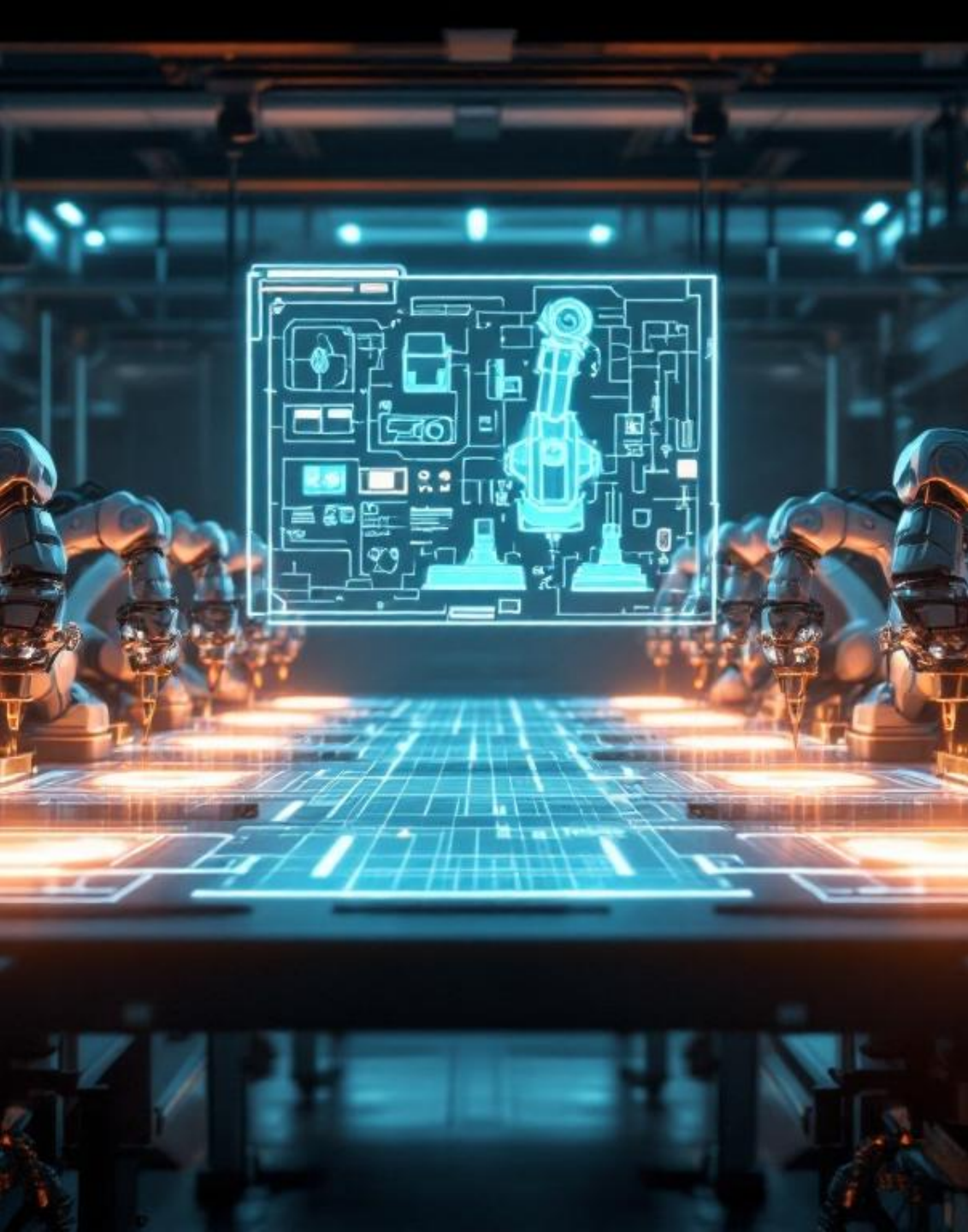
- FP32: $175B \times 4\text{bytes} \approx 700\text{GB}$
- FP16: $175B \times 2\text{bytes} \approx 350\text{GB}$

Training also requires memory for:

- gradients;
- optimizer states;
- activations;
- temporary communication buffers.

Total memory can reach multiple terabytes before sharding or offloading





Parallel Training vs. Distributed Training

Parallelism is about how you divide the compute/parameters; distribution is about where it runs and how workers communicate and stay consistent.

- Parallelism: correctness (synchronization, gradient scaling), efficiency (load balance, communication-computation overlap).
- Distribution: network bandwidth/latency, fault tolerance, elasticity, reproducibility across nodes, cluster schedulers (Kubernetes/SLURM).
- If you scale across devices, you're doing parallel training.
- If you scale across machines (with networking concerns), you're doing distributed training. Many real systems do both.

SESSION 1: CLASSICAL DISTRIBUTED TRAINING

*In data parallelism, the **batch size** is the total number of training examples processed together before the model is updated.*

*For example, if you have a total batch size of 64 and 4 devices, each device processes a slice of 16 examples. These individual slices are often called **minibatches**.*

Distributing Training Data Across Devices

Mathematical Foundation:

- In standard SGD with batch size B , the gradient update is:

$$g = \frac{1}{|B|} \sum_{xi \in B} \nabla_{\theta} L(\theta, xi)$$

- With N devices, rank k computes:

$$g_k = \frac{1}{|B_k|} \sum_{xi \in B_k} \nabla_{\theta} L(\theta, xi)$$

- The global gradient is

$$g_{global} = \sum_{k=1}^N \frac{|B_k|}{B} g_k, \quad B = \sum_{k=1}^N |B_k|$$

- For equal local batch sizes:

$$g_{global} = \frac{1}{N} \sum_{k=1}^N g_k$$

Process Overview:

- Split the dataset into non-overlapping subsets
- Perform independent forward/backward passes
- Aggregate gradients using all-reduce
- Apply the same optimizer update on every device

Data Parallelism Implementation

DDP stands for Distributed Data Parallel, a training wrapper that replicates a model on multiple devices and averages gradients across them via collectives like all-reduce. After synchronization, each replica applies the same optimizer step so all copies stay in lockstep for correct, scalable training.

Steps

1. Deterministic, non-overlapping data sharding per rank
2. Forward + backward per device independently
3. Collective gradient synchronization (all-reduce)
4. Identical parameter updates on all devices

Good practices

- Use a seeded `DistributedSampler` for dataloaders
- Overlap communication and compute via gradient bucketing
- Configure DDP for unused params when models branch (`find_unused_parameters`)
- Gradient clipping before the optimizer step
- Mixed precision: prefer BF16 if available; otherwise FP16 with dynamic loss scaling

Notes on synchronization

- Ring all-reduce minimizes bandwidth and scales with $N-1$ hops
- Ensure optimizer steps are identical across ranks after sync

Communication Optimization

BF16 is a 16-bit floating-point format that sacrifices precision for a wide dynamic range, making it stable for training neural networks. FP16 is a 16-bit format that offers higher precision than BF16 but has a narrower range, which can cause overflow or underflow issues during training.

ResNet-50 is a popular model for image recognition whose key innovation was a "shortcut" that allows the core signal, or meaningful information, to skip over layers. These shortcuts prevent the signal from getting weak and lost in deep networks, providing a direct path that makes them much easier to train accurately.

How big are the gradients?

- Gradient size = $P \text{ params} \times b \text{ bytes/param}$
- FP32 = 4 bytes; BF16/FP16 = 2 bytes

Ring all-reduce traffic per device:

- $2 \times P \times b \times (N-1)/N$

Example (ResNet-50 $\approx$ 25M params)

- FP32 gradients $\approx$ 100 MB
- $N = 8$ GPUs $\rightarrow \approx$ 175 MB traffic per device per step

Fabrics matter

- NVLink $\sim$ 300 GB/s $\rightarrow \sim$ 0.6 ms for 175 MB (idealized)
- 100 Gbps Ethernet/InfiniBand $\approx$ 12.5 GB/s $\rightarrow \sim$ 14 ms (idealized)

Make it faster

- Overlap comm with compute via gradient bucketing
- Choose bucket sizes to avoid many tiny messages
- Use BF16/FP16 gradients to halve traffic (with proper scaling)
- For small tensors, tree-based reductions can beat ring due to lower latency

In-Class Exercise 1 (10 minutes)



NVIDIA Tesla V100 - GPU computing processor
Image source: shi.com

Scenario

- Model: ResNet-50 ($P=25\text{M}$ parameters)
- Batch size: 256 (global)
- Hardware: $N=8 \times$ NVidia V100 GPUs connected with NVLink
- Assume data type: FP32 ($b=4$ bytes/param); also consider BF16/FP16 (2 bytes)

Tasks

1. Compute raw gradient tensor size: $P \times b$
2. Per-device ring all-reduce traffic: $2 \times P \times b \times (N-1)/N$
3. Estimate sync time on NVLink (300 GB/s idealized) for $N = 4, 8, 16$
4. Identify bottlenecks and propose optimizations

Hints

- ResNet-50 FP32 grads $\approx 25\text{M} \times 4 \text{ B} \approx 100 \text{ MB}$
- $N=8$ example: traffic $\approx 2 \times 100 \text{ MB} \times 7/8 \approx 175 \text{ MB}$ per device
- Time $\approx$ traffic / bandwidth (convert units); repeat for $N = 4, 8, 16$
- Consider compute time per step when judging if comm is the bottleneck

Optimization levers to discuss

- Mixed precision (BF16/FP16) halves gradient volume
- Gradient bucketing to overlap comm with backprop
- Tune bucket size: avoid tiny messages; start overlap early
- Hierarchical or tree all-reduce for latency; intra- vs inter-node paths
- Multi-node case: 100 Gbps $\approx 12.5 \text{ GB/s}$ —re-estimate times
- Optimizer/state sharding (ZeRO-style) to cut comm/state overhead

- N : the number of GPUs
- P : the number of model parameters.
- b : the number of bytes used to store each gradient value.
- $P \times b$: the total gradient size in bytes.

In-Class Exercise 1- Solution

Given

- ResNet-50 $\approx$ 25M params
- FP32 gradients = 4 bytes/param $\rightarrow$ Gradient tensor $\approx$ 25M $\times$ 4 B = 100 MB

Per-device ring all-reduce traffic = $2 \times P \times b \times (N-1)/N$

- 4 GPUs: $2 \times 100 \text{ MB} \times 3/4 = 150 \text{ MB}$
- 8 GPUs: $2 \times 100 \text{ MB} \times 7/8 = 175 \text{ MB}$
- 16 GPUs: $2 \times 100 \text{ MB} \times 15/16 \approx 187.5 \text{ MB}$

Convert the traffic into communication time: $T_{sync} \frac{\text{traffic}}{\text{bandwidth}}$

T_{sync} : synchronization time (the estimated time required for all GPUs to exchange and combine their gradients during the all-reduce operation).

Idealized NVLink sync time (300 GB/s)

- 4 GPUs: $150 \text{ MB} / 300 \text{ GB/s} \approx 0.50 \text{ ms}$
- 8 GPUs: $175 \text{ MB} / 300 \text{ GB/s} \approx 0.58\text{--}0.60 \text{ ms}$
- 16 GPUs: $187.5 \text{ MB} / 300 \text{ GB/s} \approx 0.62\text{--}0.63 \text{ ms}$

Insights

- Counterintuitive: per-device traffic increases slightly with N in ring ((N-1)/N term)
- On fast intra-node links, sync looks tiny; on 100 Gbps (12.5 GB/s), times jump to 12–15 ms
- Mitigations: overlap with backprop (bucketing), lower precision (BF16/FP16), hierarchical/tree collectives, optimizer/state sharding

Model Parallelism Strategies

When to use

- Model doesn't fit on one device (memory)
- Boost throughput when data parallelism alone saturates

Layer/pipeline parallelism

- Partition consecutive layers into K stages placed on different devices
- Forward: activations flow stage 1 $\rightarrow$... $\rightarrow$ K
- Backward: gradients flow K $\rightarrow$... $\rightarrow$ 1

Keep the pipeline full

- Split each global batch into M microbatches
- While stage s handles microbatch i, stage s+1 handles i-1, etc.
- Aim: minimize “bubbles” (idle time) at pipeline start/end

Practical goals and knobs

- Balance stage runtimes (group light layers together; profile and iterate)
- Manage activation memory: activation checkpointing trades extra compute for lower memory
- Choose M (microbatch count) to trade utilization vs. latency/memory
- Overlap inter-stage communication with compute when possible
- Combine with data parallelism for scale (hybrid parallelism)



Pipeline Parallelism: Utilization vs. Memory

Let:

- K = number of pipeline stages
- M = number of microbatches

For balanced stages, approximate pipeline utilization is:

$$Utilization \approx \frac{M}{M + K - 1}$$

So, the bubble fraction is:

$$Bubble\ fraction \approx \frac{K - 1}{M + K - 1}$$

When $M \gg K$:

$$Bubble\ fraction \approx \frac{K - 1}{M}$$

Schedulers

- GPipe: all forward passes, then all backward passes
 - simple and consistent;
 - high activation-memory cost.
- 1F1B: interleaves one forward and one backward pass
 - lower activation-memory cost;
 - better utilization after warm-up.



Hybrid Parallelism Design



Combine parallelisms

- Inside a node/pod: tensor or pipeline (and/or layer) parallel to fit/accelerate the model
- Across nodes: data parallel replicas for scale

Hierarchical communication

- Intra-replica (within a model shard/pipeline/tensor group): fast NVLink/NVSwitch collectives with high frequency
- Inter-replica (across data-parallel groups): less frequent gradient/optimizer aggregations over slower links

Example layout: 32 GPUs

- 4-way model/tensor parallel $\times$ 8-way data parallel = 32 ranks
- Each DP group contains 4 GPUs that form one full model via tensor/pipeline splits

Topology-aware mapping

- Keep tensor/pipeline group members on the same node/NVSwitch island
- Place DP groups to minimize cross-switch hops for all-reduce
- Align process ranks with fabric (NUMA, NIC, switch)

Optimizer and memory

- Add optimizer/state sharding (ZeRO/FSDP) atop model+data parallel to cut memory
- Prefer local, intra-node shards/collectives when possible

Operational tips

- Use hierarchical all-reduce: intra-node reduce + inter-node reduce + intra-node broadcast
- Tune microbatch count (pipeline) and bucket sizes (DP) for overlap
- Monitor imbalance: rebalance pipeline stages and tensor partitions

Scalability Analysis

Sources of scaling loss

- Communication overhead: bandwidth limits, latency, protocol inefficiency
- Synchronization barriers and stragglers: everyone waits for the slowest
- Load imbalance/heterogeneity: uneven work, throttling, ECC, I/O skew
- Memory pressure: activations/optimizer state constrain batch/parallelism

Measuring efficiency

- Ideal speedup = N
- Actual speedup = T_1 / T_N
- Parallel efficiency = $(T_1 / T_N) / N$

Amdahl's Law (upper bound on speedup)

- Speedup $\leq 1 / (s + (1 - s)/N)$, where s = serial fraction
- Example: $s = 0.10 \Rightarrow \text{max speedup} = 10\times \text{ as } N \rightarrow \infty$
- Even small s severely caps efficiency at large N

What to do (reduce s , hide comm)

- Overlap communication with compute: gradient bucketing, fused kernels
- Hierarchical collectives; topology-aware placement
- Lower-precision or compressed gradients with error feedback
- Algorithmic options: increase local batch; partial/delayed sync; asynchronous variants with care
- Balance work: uniform microbatching, stage rebalancing, dynamic batch assignment
- Detect and mitigate stragglers: health checks, retry, prefetch, pinned workers

Distributed Systems Concerns in Practice

Fabrics and topology:

- Intra-node: PCIe, NVLink
- Inter-node: 25/100/200/400 Gbps Ethernet/InfiniBand; latencies 2–10× higher

Collectives vs parameter server:

- Collectives (NCCL/Gloo/MPI): bandwidth-efficient, symmetric
- PS: flexible, supports async, but can bottleneck at servers

Orchestration & launch:

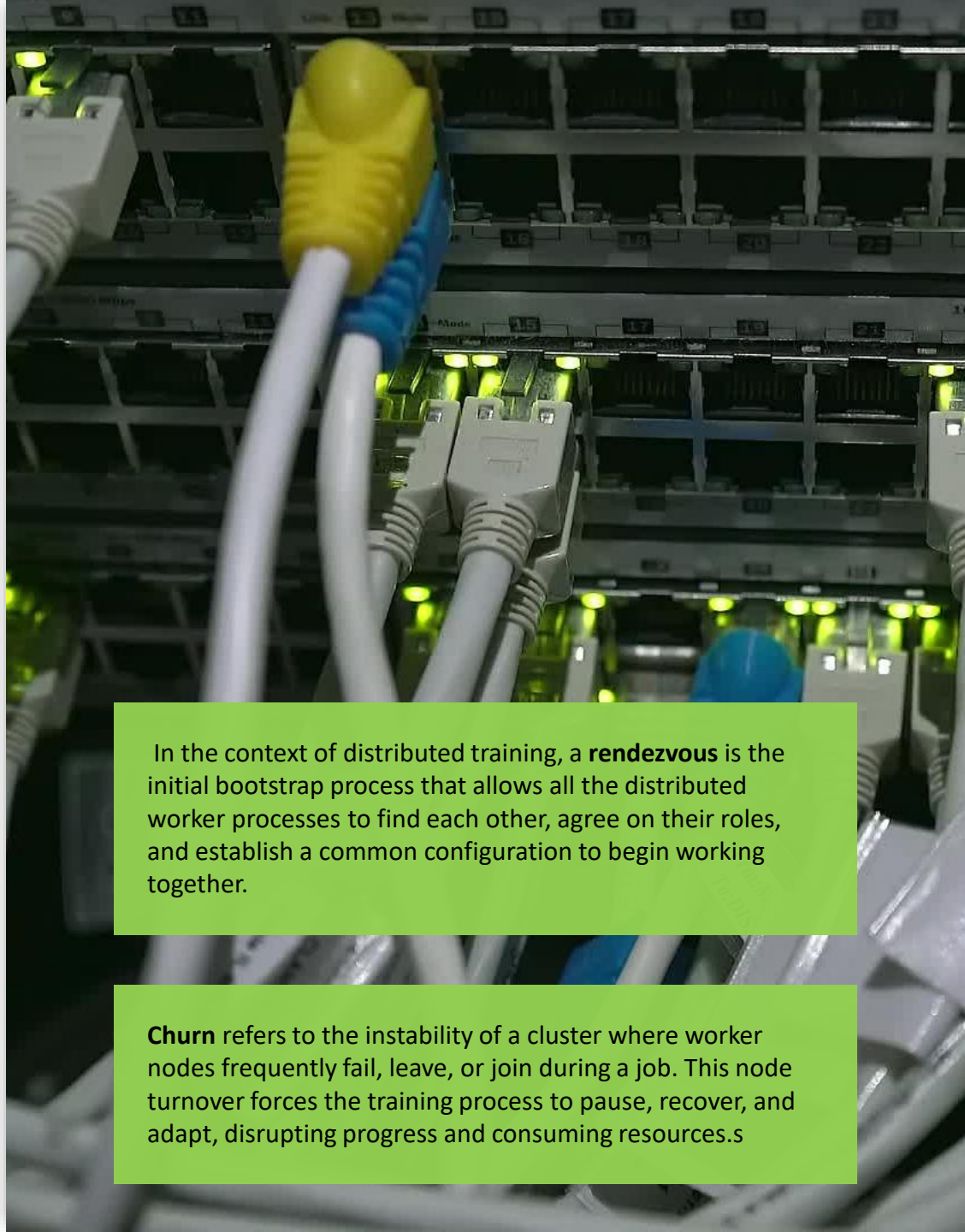
- SLURM/Kubernetes; ranks, world size, rendezvous; topology-aware placement

Reproducibility:

- Deterministic sharding and seeding; avoid nondeterministic kernels

Elasticity:

- Handle node churn; reshard data and adjust LR safely



In the context of distributed training, a **rendezvous** is the initial bootstrap process that allows all the distributed worker processes to find each other, agree on their roles, and establish a common configuration to begin working together.

Churn refers to the instability of a cluster where worker nodes frequently fail, leave, or join during a job. This node turnover forces the training process to pause, recover, and adapt, disrupting progress and consuming resources.

Sync vs Async vs Elastic Data Parallel

Synchronous all-reduce:

- Pros: stable convergence; single-node equivalence
- Cons: straggler-sensitive; per-step barrier

Parameter-server (async/stale-sync):

- Pros: more throughput; tolerant to slow clients
- Cons: stale gradients; careful LR tuning; potential accuracy hit

Elastic DP (synchronous with changing world size):

- World size adjusts at runtime; reshard data/checkpoints; adjust LR with effective batch

Guidance: prefer sync for accuracy-critical; use async/elastic when churn is inevitable

Topology-Aware & Hierarchical All-Reduce; Libraries

- Libraries:
 - NCCL (GPU-centric), Gloo (CPU/GPU), MPI (HPC)
- Algorithms:
 - Ring: bandwidth-optimal for large messages
 - Tree: lower latency for smaller messages
 - Reduce-scatter + all-gather: core of sharded training
- Hierarchical all-reduce:
 - Intra-node reduce-scatter (NVLink)
 - Inter-node reduce-scatter (IB/Ethernet)
 - Inter-node all-gather
 - Intra-node all-gather
- Practice: bucket sizes, algorithm selection per tensor size, topology hints

Tensor Parallelism & Sharding (Megatron, ZeRO/FSDP)

Tensor parallel (intra-layer):

- Split attention heads or MLP dimensions across GPUs
- Per-layer collectives: all-gather/all-reduce or reduce-scatter in fwd/bwd
- Works best within fast NVLink/NVSwitch “islands”

Optimizer/gradient/parameter sharding:

- Stage 1: shard optimizer states
- Stage 2: shard optimizer + gradients
- Stage 3: shard optimizer + gradients + parameters (optionally offload to CPU/NVMe)

Reduces per-rank memory; introduces extra comm for gathers/scatters

FSDP (PyTorch):

- Fully sharded per module; parameters/optimizer states are sharded
- All-gather param shards before compute; reduce-scatter grads after
- Configurable wrap/granularity, mixed precision, and offload

Trade-off:

- Big memory relief enables very large models
- More frequent and fine-grained communication—tune bucket sizes, prefetch, and overlap
- Combine with activation checkpointing and BF16/FP16



Reproducibility & Determinism at Scale

Seeds and sharding

- Set global and per-rank RNG seeds (Python, NumPy, framework, CUDA)
- Use a deterministic DistributedSampler so each rank gets a stable, unique shard

Deterministic ops

- Enable deterministic backends/flags
- Avoid or replace known nondeterministic kernels; document any exceptions

Floating-point non-associativity

- Different reduction orders/bucketization yield small numeric diffs
- Validate with tolerances (e.g., atol/rtol), not strict bitwise equality

Version and environment pinning

- Record container image, CUDA/cuDNN/NCCL, driver, framework versions
- Log code commit, configs/hyperparams, hardware topology and world size

Checkpoints

- Save model weights, optimizer state, RNG states (CPU/CUDA), and dataloader epoch/offset
- Include sharding metadata (DP/TP/PP groups, FSDP/ZeRO maps) for exact restarts
- Support elastic/changed world sizes with remapping info

Fault Tolerance, Checkpointing & Orchestration

Checkpointing:	- Save model, optimizer, LR scheduler, RNG states (CPU/CUDA), and sharding/group metadata - Write atomically to durable storage; verify and keep rolling N checkpoints
Failure handling:	- Synchronous jobs: relaunch and load latest checkpoint - Elastic jobs: surviving ranks continue; newcomers rendezvous, rebuild groups, and reshard from checkpoint
Launch/orchestration:	- Use SLURM/Kubernetes operators; set rank/world-size via env - Topology-aware placement (keep tightly coupled groups on same node/switch) - Add health checks, timeouts, exponential backoff, and automatic restart policies
LR & batch when elastic:	- Recompute effective global batch after world-size changes - Adjust learning rate accordingly; maintain gradient scaling and loss-scaling consistency - Log events and validate convergence after elastic resizes

SESSION 2: FEDERATED LEARNING SYSTEMS

What is Federated Learning

- Decentralized training across many edge devices or clients
- Data stays on device; only model updates or gradients are shared

Why it matters

- Privacy preservation and regulatory compliance
- Lower central bandwidth use; clients compute locally

Core challenges

- Clients are intermittent and heterogeneous (CPU/GPU, battery, network)
- Non-IID data across clients; drift and imbalance
- Faults and stragglers; limited communication rounds

FedAvg (Federated Averaging) protocol

- Round t :
 - Server broadcasts current model to selected clients
 - Clients train locally for a few epochs on their data
 - Clients send model deltas or updated weights to server
 - Server aggregates (weighted average by client data size) $\rightarrow$ new global model

Practical tips

- Client sampling: pick a subset each round to scale and reduce stragglers
- Weighting: average by number of local examples to reduce bias
- Compression: quantize/sparsify updates to save bandwidth
- Security/privacy: secure aggregation, differential privacy, device attestation
- Evaluation: track both global and per-client metrics; handle skewed datasets

Client Scheduling and Participation

Why scheduling matters

- Clients are heterogeneous and intermittent; naïve selection creates bias and slow rounds

Eligibility criteria (examples)

- Sufficient battery and charging state
- Unmetered/high-quality network; device idle and not in active use
- Minimum storage/CPU/GPU/RAM; recent app/version; security/attestation passed

Selection strategies

- Random sampling from eligible pool each round
- Stratified sampling by geography, device class, or data attributes to reduce bias
- Priority/credit systems to ensure fairness and fresh participation
- Cap per-client frequency to avoid overusing fast devices

Availability patterns to consider

- Time zones and diurnal patterns
- Usage behavior (work hours vs. overnight charging)
- Network variability by region/carrier; hardware capability differences

Robustness and performance

- Over-select (invite $K \gg \text{target}$) to tolerate dropouts; accept first N that check in
- Deadlines and partial aggregation; early stopping if enough updates arrive
- Adaptive cohort size: increase when stragglers are common
- Secure eligibility checks; anonymized telemetry for scheduling
- Metrics to monitor
- Round completion time, acceptance rate, dropout rate, coverage/fairness by strata
- Contribution quality/weight (examples seen, loss improvement)

Communication- Efficient Protocols

Core idea: cut bytes per round without hurting accuracy

Techniques

- **Quantization:** send updates in 8/4/2 bits with per-tensor scales
- **Sparsification:** top-k or thresholded updates; error feedback to recover dropped mass
- **Structured updates:** layer-wise or last-layer only; low-rank adapters (LoRA/PEFT)
- **Delta/periodic sync:** send changes since last round; skip small updates

Practicalities

- Clip and add DP noise if required; secure aggregation friendly
- Calibrate k/bit-width; monitor loss drift and add error feedback
- Compress after aggregation when possible to reduce server load

In class Quiz 1

Question 1

What is the primary reason the keyboard app uses federated learning?

- A. To speed up the server's training loop
- B. To avoid sending raw user text off the device
- C. To make models larger without extra cost
- D. To reduce the number of users needed

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Answer 1

B. To avoid sending raw user text off the device

- Why B is correct: Federated learning keeps data on-device and shares only updates, aligning with the privacy goal.
- Why others are not: A and C aren't inherent benefits; D is unrelated.

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Question 2

When should local training be triggered on a user's device?

- A. Anytime the user is actively typing
- B. Only when the phone is idle, charging, on Wi-Fi, and has sufficient battery
- C. Whenever the app is open, regardless of conditions
- D. Only during OS updates at night

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Answer 2

B. Only when the phone is idle, charging, on Wi-Fi, and has sufficient battery

- Why B is correct: The scenario emphasizes resource-friendly conditions to avoid user disruption.
- Why others are not: A and C risk interference and battery drain; D is an arbitrary schedule not stated.

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Question 3

Why consider selecting clients with stratified sampling across language, device type, and usage pattern?

- A. To ensure fair representation across diverse segments
- B. To minimize the total number of participants
- C. To guarantee only high-end devices participate
- D. To select only the fastest typists

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Answer 3

A. To ensure fair representation across diverse segments

- Why A is correct: Stratification helps the global model generalize across differing user groups.
- Why others are not: B, C, and D bias the sample and undermine representativeness.

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Question 4

What trade-off is addressed by sharing only the final prediction layer (e.g., vocabulary-related weights)?

- A. Privacy vs. encryption strength
- B. Personalization and communication efficiency vs. full-model synchronization
- C. Model size vs. server memory limits
- D. Latency vs. screen size

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Answer 4

B. Personalization and communication

- Why B is correct: Sending a smaller subset reduces bandwidth and preserves on-device personalization in lower layers.
- Why others are not: A, C, and D don't capture this design choice.

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Question 5

The team targets about a 90% reduction in update size via compression. What does this imply?

- A. Each update is roughly 10% of its original size
- B. Only 10% of users will train
- C. The model's accuracy will drop by exactly 10%
- D. 10% fewer training rounds are needed

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Answer 5

A. Each update is roughly 10% of its original size

- Why A is correct: A 90% reduction means payloads shrink to about one-tenth.
- Why others are not: Participation, accuracy, and rounds are not determined by the compression figure alone.

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Question 6

What purpose does adding calibrated Gaussian noise to gradients serve?

- A. It regularizes the model to prevent overfitting only
- B. It compresses the gradients further
- C. It provides differential privacy by masking any single user's influence
- D. It guarantees zero privacy leakage

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Answer 6

C. It provides differential privacy by masking any single user's influence

- Why C is correct: DP noise limits how much one user's data can alter the shared update.
- Why others are not: A is a side effect at best; B is different from noise; D is too strong—DP offers bounded risk, not absolute zero.

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Question 7

Which parameter directly governs the privacy–utility trade-off in the DP mechanism described?

- A. Learning rate
- B. Epsilon (privacy budget)
- C. Batch size on device
- D. Number of training rounds

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Answer 7

B. Epsilon (privacy budget)

- Why B is correct: Epsilon determines the required noise scale for DP.
- Why others are not: They affect training dynamics, not the DP guarantee itself.

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Question 8

Why is it important to include different device types (phones and tablets) in client selection?

- A. Because tablets never experience battery drain
- B. To ensure the model learns from varied input behaviors and hardware contexts
- C. To simplify server-side aggregation
- D. To avoid applying differential privacy

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Answer 8

B. To ensure the model learns from varied input behaviors and hardware contexts

- Why B is correct: Different devices can imply distinct usage patterns and text behaviors; representation improves generalization.
- Why others are not: A is false; C and D are unrelated.

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Question 9

What is a likely benefit of keeping most layers local and only sending the output layer updates?

- A. Users can edit the model architecture manually
- B. Faster round times due to smaller payloads and less uplink bandwidth
- C. The server no longer needs to aggregate updates
- D. No need to schedule training opportunistically

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Answer 9

B. Faster round times due to smaller payloads and less uplink bandwidth

- Why B is correct: Smaller updates reduce communication time, improving round efficiency.
- Why others are not: A and C are incorrect; D remains necessary to protect user experience.

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Question 10

Why is sensitivity considered when computing the noise scale for DP?

- A. To match the server's CPU capacity
- B. To calibrate noise to the maximum effect a single user's data could have on the update
- C. To ensure compression algorithms work properly
- D. To fix model convergence speed

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

In class Quiz 1

Answer 10

B. To calibrate noise to the maximum effect a single user's data could have on the update

- Why B is correct: DP noise is tied to the sensitivity of the function to bound any one user's influence.
- Why others are not: A, C, and D are unrelated to the privacy guarantee.

Scenario: Federated Smart Keyboard Prediction

A mobile keyboard app wants to improve next-word prediction without uploading raw user text. The app will use federated learning: each phone trains a local language model on its typing sequences and sends only model updates to a central server.

To protect privacy, updates may include differentially private noise. Training should only run when it won't bother the user or drain resources—for example, while the phone is idle, charging, connected to Wi-Fi, and has sufficient battery.

Because the user base is diverse, the team must select clients each round so that languages (e.g., English, Spanish, Chinese), device types (phones vs. tablets), and usage patterns (heavy, moderate, light typists) are all represented.

To cut bandwidth, the team considers sharing only the final prediction layer (e.g., vocabulary logits/embeddings) and compressing those updates (such as with quantization), targeting about a 90% reduction in payload size.

Differential privacy will be applied by adding calibrated Gaussian noise to local gradients, with a noise scale computed from a chosen privacy budget epsilon and the sensitivity of the update.

The design must balance communication cost, privacy protection, model quality, and personalization.

Federated Personalization

Goal: balance one global model with user-specific adaptation

Strategies

- Local fine-tuning: keep a private head or small layers and adapt on-device
- Split models: shared backbone + personal head; only backbone is aggregated
- Meta/regularized training: optimize global loss + personalization term

Practical tips

- Cache small personal params; don't upload them
- Control drift with lightweight regularization and early stopping
- Evaluate both global and per-segment metrics



Non-IID Data Challenges

What's hard

- Client data are skewed and imbalanced; distributions drift over time

Diagnose

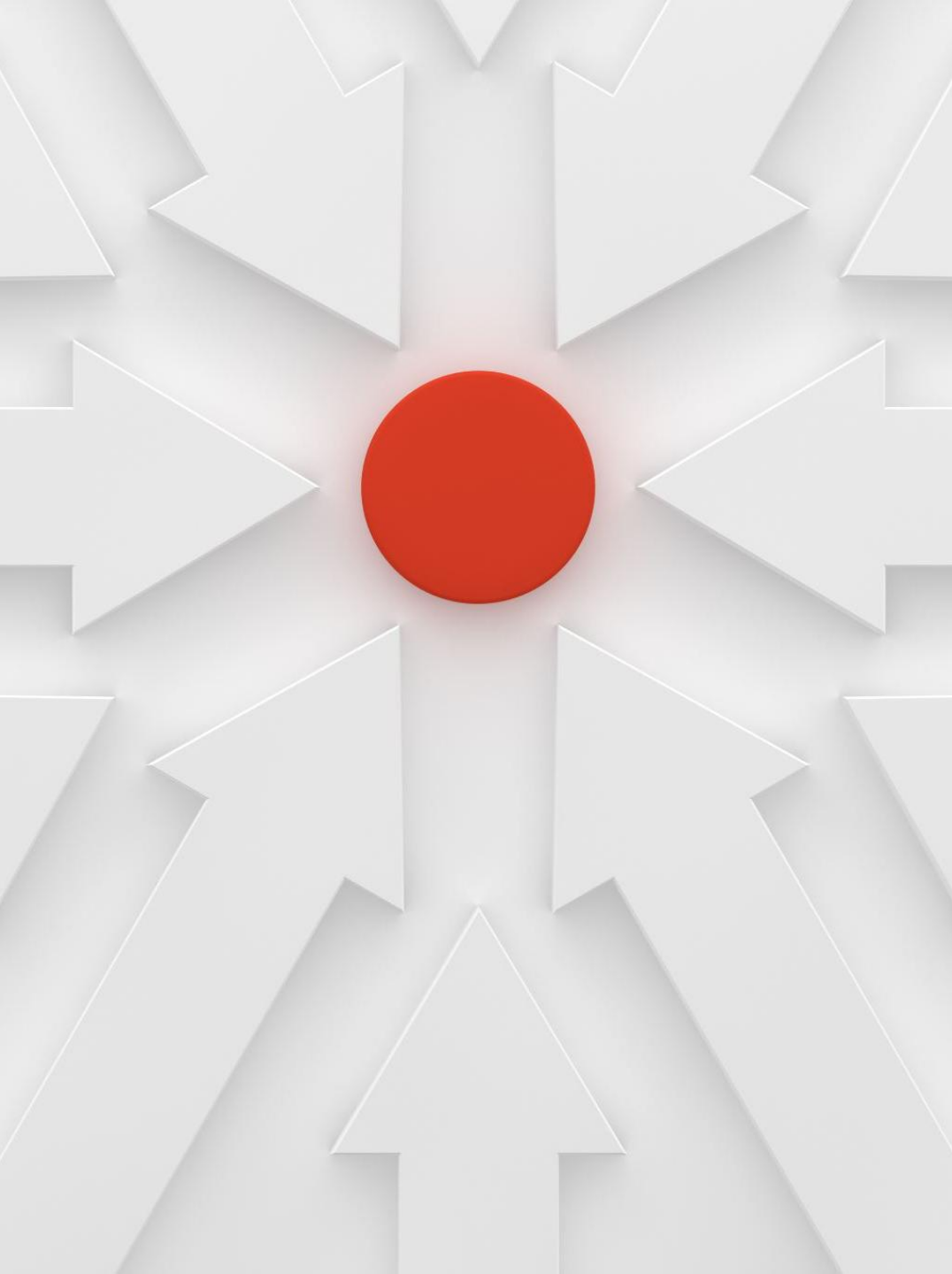
- Compute per-client stats (label/feature histograms, entropy, KS tests)
- Cluster similar clients; track drift across rounds

Adapt aggregation

- Weight by data size, quality, and similarity to global
- Use cohort-wise or personalized aggregation instead of one-size-fits-all

Practical guardrails

- Robustness: clip norms, outlier filtering
- Evaluation: group metrics and worst-case performance



Federated Learning Performance

Convergence factors

- Local steps vs. rounds; data and system heterogeneity; client sampling

Efficiency trade-offs

- More local steps → fewer rounds but risk of drift
- Communication cost vs. accuracy curve

Key metrics

- Rounds to target accuracy; bandwidth per round/total
- Participation rate and straggler impact
- On-device energy/time per round

SESSION 3: PRIVACY, SECURITY & DEPLOYMENT

By the end of this session, we should be able to:

- distinguish data locality, confidentiality, and formal privacy;
- explain the separate roles of differential privacy, secure aggregation, and encryption;
- identify privacy and integrity threats in federated systems;
- design deployment controls for heterogeneous, unreliable, and regulated environments.

Key idea: Federated learning keeps raw data local, but it does not automatically eliminate privacy leakage, attacks, or compliance obligations.



Training
collaboratively
doesn't
automatically
guarantee privacy

Threats

- Gradient/model inversion (reconstruct inputs)
- Membership inference (was a record in training?)
- Property inference and data poisoning/backdoors

Risk factors

- Small batches, overfitting, rich gradients, text/images

Defenses

- Secure aggregation + encryption in transit/at rest
- Differential privacy with clipping + noise
- Regularization, larger batches, early stopping; audit and red-team



Differential Privacy Mechanisms

- **Definition**
 - (ϵ, δ) -DP: output barely changes when one record is added/removed
- **Core ingredients**
 - Clip per-client update to bound sensitivity
 - Add calibrated Gaussian noise; track privacy with an accountant
- **In federated learning**
 - Do clipping + noise on device; combine with secure aggregation
 - Monitor spent ϵ, δ ; stop or adjust when budget nears limit
- **Practical tuning**
 - Set clip norm from quantiles; choose noise to hit target ϵ
 - Larger batches and fewer rounds reduce privacy cost

Secure Aggregation Protocols

Purpose: server learns only the sum of client updates—never any individual update

Approaches

- Masking with secret sharing (pairwise masks cancel in the sum; threshold recovery)
- Homomorphic encryption (server aggregates ciphertexts)

Practical notes

- Handle dropouts with threshold schemes and mask-recovery
- Combine with DP; encrypt in transit and at rest
- Validate with audits and fault-injection tests



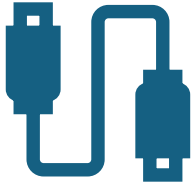
Deployment Challenges

Heterogeneous devices

- Wide ranges in compute, memory, storage, battery, connectivity

Adaptive strategies

- Tune local steps, batch size, precision (quantization), and cadence per device tier
- Opportunistic scheduling: train on charge + Wi-Fi; pause on battery/roaming
- Compress updates: sparsification, quantization, delta encoding



Networking realities

- Handle dropouts; resumable uploads; partial aggregation with quorum
- Rate limit and backoff; prioritize low-latency cohorts separately

Ops and safety

- Versioning and rollout gates; A/B by cohort
- Telemetry with privacy: success/failure codes, energy/time, bytes
- Fallbacks to disable training if heat, battery, or data caps trigger

Fault Tolerance and Reliability

Participation churn

- Over-invite; accept first N updates; retry/backoff; resumable uploads

Asynchronous/partial aggregation

- Quorum-based rounds; rolling aggregation; staleness-aware weighting



Staleness management

- Tag updates with model version; decay weights by lag; bound max staleness

Robustness to bad/malicious clients

- Norm clipping, validation-on-server, robust aggregators (median/trimmed mean)
- Outlier and consistency checks; reputation/blacklisting

Resilience engineering

- Idempotent jobs, checkpointing, rate limiting, circuit breakers
- Telemetry/alerts for dropout spikes and drift

A **quorum** is the minimum number of clients that must respond before the server proceeds with a training round. Instead of waiting for every single client, which could take forever, it aggregates updates once this representative group has reported in.

Performance Optimization

Smart client selection

- Score by data utility, device health, connectivity; diversify to avoid bias

Communication scheduling

- Prefer Wi-Fi + charging; predict availability windows; cache and batch updates

Adaptive training

- Per-device configs: batch size, local steps, precision; auto-tune online
- Early exit for slow/overheating devices

Bandwidth/compute efficiency

- Update compression: sparsify, quantize, delta + server-side momentum
- Partial model updates or adapters; server-side aggregation acceleration

Energy awareness

- Train only on charge or high battery; throttle CPU/GPU; budget per session

Metrics to track

- Time/round, completion rate, bytes/round, on-device energy, accuracy/round



In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Question 1

What is the main reason these hospitals use federated learning?

- A. To merge all raw datasets into a central data lake
- B. To train a shared model without moving raw patient data off premises
- C. To reduce the number of participating hospitals
- D. To avoid any need for encryption

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Answer 1

B. To train a shared model without moving raw patient data off premises

- Why B is correct: Federated learning keeps raw data local while enabling collaborative training.
- Why others are not: A contradicts privacy goals; C is irrelevant; D is false—encryption is still valuable.

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Question 2

Which preprocessing step best reduces re-identification risk?

- A. Scaling features to zero mean and unit variance
- B. Duplicating rare patient cases
- C. Replacing exact age with brackets and using categorical symptom indicators
- D. Adding more exact timestamps

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Answer 2

C. Replacing exact age with brackets and using categorical symptom indicators

Coarsening quasi-identifiers reduces identifiability. Others don't, and some increase risk.

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Question 3

Why is secure aggregation useful here?

- A. It compresses model updates
- B. It lets the coordinator see only aggregates, not individual hospital updates
- C. It replaces authentication
- D. It guarantees identical data distributions

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Answer 3

B. It lets the coordinator see only aggregates, not individual hospital updates

Secure aggregation hides individual updates. Others are unrelated or impossible.

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Question 4

Differential privacy most directly aims to:

- A. Limit how much any single patient's data can influence a shared update
- B. Prevent packet loss on hospital networks
- C. Ensure accuracy increases every round
- D. Equalize performance across specialties

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Answer 4

A. Limit how much any single patient's data can influence a shared update

DP bounds individual influence. Others are not DP guarantees.

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Question 5

Which parameter governs the privacy–utility trade-off for DP?

- A. Number of hospitals
- B. Batch size
- C. Epsilon (privacy budget)
- D. Learning rate

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Answer 5

C. Epsilon (privacy budget)

Epsilon sets noise magnitude. Others affect training dynamics, not the DP guarantee.

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Question 6

Why might noise be added per feature based on sensitivity?

- A. To match server CPU throughput
- B. Because different features can reveal different amounts of information
- C. To keep gradients exactly zero in expectation
- D. To rotate encryption keys

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Answer 6

B. Because different features can reveal different amounts of information

Sensitivity-aware calibration tailors protection. Others are unrelated.

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Question 7

Hospital heterogeneity primarily impacts which design aspect?

- A. Keyboard layout on staff devices
- B. The aggregation method and weighting/fairness of updates
- C. The need for TLS
- D. Whether audit logs are necessary

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Answer 7

B. The aggregation method and weighting/fairness of updates

Non-IID data requires careful aggregation.
Others are unaffected.

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Question 8

Which control best supports regulatory compliance?

- A. Anonymous public access to model checkpoints
- B. Training only during off-peak hours
- C. Role-based access control with comprehensive audit logging
- D. Disabling consent prompts to streamline workflow

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Answer 8

C. Role-based access control with comprehensive audit logging

RBAC and auditing are core compliance measures. Others violate or don't address compliance.

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Question 9

What does a "right to be forgotten" mechanism require?

- A. Deleting the global model entirely
- B. Broadcasting raw data so others can verify deletion
- C. Converting models to rule-based systems
- D. Ability to remove/exclude a patient's data from future training

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Answer 9

D. Ability to remove/exclude a patient's data from future training

Compliance requires excluding an individual's data going forward. Others are incorrect or unsafe.

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Question 10

Why is end-to-end encryption necessary even with secure aggregation and DP?

- A. To speed up aggregation computations
- B. Because encryption alone provides DP
- C. To protect updates in transit and at rest from network/storage adversaries
- D. To make audit logging unnecessary

In class - Quiz 2

Scenario: Privacy-Preserving Federated Learning for Healthcare

Several hospitals want to collaboratively train a medical diagnosis model without sharing raw patient data. Each hospital keeps its data in-house and participates in federated learning by training locally and sending only model updates to a coordinator.

Because medical data is highly sensitive, the design layers multiple privacy measures: data is preprocessed to reduce identifiability (for example, replacing exact ages with age brackets and using categorical indicators instead of detailed symptoms), local updates can be protected with differential privacy by adding calibrated Gaussian noise per feature based on its sensitivity and a chosen privacy budget (epsilon), and transport/storage use end-to-end encryption.

For added security, the system supports secure aggregation so the coordinator can only see aggregated updates, not any individual hospital's update. Hospitals differ in size and specialty (e.g., cardiology, oncology, pediatrics), so the aggregation process must handle heterogeneous data distributions.

The deployment must comply with regulations (such as HIPAA-like requirements), including role-based access controls, audit logging of all data access, explicit patient consent for federated learning, and a mechanism to delete or exclude a patient's data from future model updates ("right to be forgotten").

The goal is to balance model utility with strong privacy, security, and compliance across diverse healthcare environments.

Answer 10

C. To protect updates in transit and at rest from network/storage adversaries

Encryption mitigates transport/storage threats; it doesn't replace DP or auditing.



Privacy Design Document Framework

Systematic:

Privacy analysis for deployment

Data Collection Assessment:

What data, why it's needed, how gathered; include sensor data, interactions, on-device derivations

Data Usage Documentation:

How data is processed; whether used for training; transformations before leaving device; prefer on-device processing

Data Sharing and Storage Policies:

Who has access; retention and deletion conditions; treat aggregated updates as potentially sensitive

Consent and Transparency:

Explain what's collected and benefits; go beyond legal compliance with clear communication of value and trade-offs



Deployment Security Considerations

Protecting Systems and Users

Physical Security Challenges:

- Embedded devices are physically accessible; require secure boot and encrypted storage

Model Protection Strategies:

- Full protection is impossible; use obfuscation and on-device key storage to deter casual attacks

Over-the-Air Update Risks:

- OTA expands attack surface; avoid unless essential, weigh security vs. benefit for embedded ML

Network Security Design:

- Minimize exposed services; prefer transmit-only clients; use secure protocols for data in transit

Deployment Validation:

- Dog-food and field-test under supervision to uncover issues lab tests miss

Summary - Week 5 Key Takeaways

Core Distributed Training Techniques:

- Classical approaches: Data parallelism scales datasets, model parallelism handles large models
- Hybrid strategies: Combine approaches for maximum scalability and efficiency
- Communication optimization: Compression and efficient protocols reduce bandwidth requirements
- Federated learning: Privacy-preserving distributed training for edge devices

System Design Principles:

- Privacy by design: Implement multiple layers of privacy protection
- Fault tolerance: Handle device failures and malicious participants gracefully
- Adaptive protocols: Adjust to device capabilities and network conditions
- Security first: Protect against reconstruction and inference attacks

Real-world Deployment Considerations:

- Hardware heterogeneity requires adaptive training strategies
- Network conditions affect communication protocol design
- Regulatory compliance adds complexity but ensures responsible deployment
- Performance optimization balances accuracy, privacy, and efficiency

Looking Ahead - Week 6 Preview

Hardware Backend Specialization

- **Next Week's Focus:**
 - Hardware spectrum from data center accelerators to microcontrollers
 - Hardware-aware model design and optimization strategies
 - Specialized processors and their computational trade-offs
 - Backend-specific optimization techniques
- **Connection to Distributed Training:**
 - Distributed systems must account for heterogeneous hardware capabilities
 - Communication protocols depend on available hardware features
 - Model parallelism strategies vary by accelerator architecture
 - Edge federated learning requires ultra-efficient hardware utilization
- **Required Readings for Week 6:**
 - [Book 1 - Volume 1 — Hardware Acceleration](#)
 - [Book 1 - Volume 1 — Model Compression](#)
 - [Book 1 - Volume 2 — Edge Intelligence](#)
 - [Book 2: Chapter 2: Getting Started \(hardware overview\)](#)
 - [Book 2: Chapter 14: Designing Your Own TinyML Applications](#)